

Софийски университет “Св. Климент Охридски”

Факултет по математика и информатика

Предмет: Въведение в дълбокото самообучение

Тема: 3. Семантично сегментиране на СНИМКИ

Проектна документация

Изготвил: Явор Чамов

Факултетен номер: 4MI3400787

Финален модел: FPN + EfficientNet-B3 + CE Dice

27 юни 2026 г.

Съдържание

1	Резюме	4
2	Въведение	4
3	Цел на проекта и изисквания	4
4	Dataset: Cityscapes	5
4.1	Използвани пакети	5
4.2	Използване на split-овете	6
4.3	Класове	6
5	Exploratory Data Analysis и подготовка на данните	6
5.1	Preprocessing	7
5.2	Augmentation стратегии	7
5.3	EDA визуализации от reports/figures	8
6	Training pipeline	13
7	Използвани архитектури	14
7.1	Baseline majority-class model	14
7.2	Tiny U-Net	15
7.3	U-Net	15
7.4	U-Net++	15
7.5	FPN	16
7.6	PSPNet	16
7.7	DeepLabV3+	16
7.8	SegFormer	16
8	Използвани encoders	17
8.1	ResNet	17
8.2	EfficientNet-B3	17

8.3	MiT-B1	17
9	Loss функции	17
9.1	Cross Entropy Loss	17
9.2	Dice Loss	18
9.3	Focal Loss	18
9.4	Lovasz Loss	18
10	Метрики за оценка	18
10.1	Pixel Accuracy	18
10.2	Intersection over Union	19
10.3	Dice coefficient	19
11	Експериментален план	19
12	Model report и резултати	19
13	Избор на най-добър модел	22
14	Анализ на top моделите	23
14.1	FPN + EfficientNet-B3	23
14.2	DeepLabV3+ + EfficientNet-B3	23
14.3	SegFormer MiT-B1	23
14.4	U-Net и U-Net++	23
15	Loss curves и generalization gap	23
16	Confusion matrix и error analysis	24
16.1	Qualitative validation examples	29
17	Streamlit demo приложение	30
18	Тестове и качество на кода	30
19	Ограничения	31

20	Бъдеща работа	31
21	Заключение	32
А	Пълен списък на експериментите	33
Б	Примерна структура на проекта	35
В	Използвана литература	35

1 Резюме

Настоящият документ описва цялостната реализация на проект за **семантично сегментиране** върху **Cityscapes Dataset**. Целта е да се обучат и сравнят различни невронни архитектури, които приемат RGB изображение от градска сцена и връщат маска, в която всеки пиксел е класифициран към един от 19 семантични класа.

Проектът следва пълен работен процес: подготовка на данни, exploratory data analysis, имплементация на training pipeline, сравнение на архитектури, loss функции и learning-rate schedulers, изграждане на model report и Streamlit demo приложение. Основният план за реализация включва изисквания за Cityscapes dataset, сравнение на Cross Entropy, Dice и Focal loss, експерименти с autoencoder/U-Net тип мрежа, cosine annealing и step decay scheduler-и, Streamlit интерфейс и tests.

Финално най-добър модел е избран `046_fpn_efficientnet_b3_ce_dice`, който използва **FPN архитектурата**, **EfficientNet-B3 encoder**, ImageNet pretrained weights, **Cross Entropy + Dice loss** и **Cosine Annealing scheduler**. Моделът постига:

- **mean IoU** = 0.7283;
- **mean Dice** = 0.8335;
- **pixel accuracy** = 0.9503.

2 Въведение

Семантичното сегментиране е задача от компютърното зрение, при която всеки пиксел от изображението се класифицира към предварително дефиниран клас. За разлика от image classification, където се предсказва един клас за цялото изображение, и object detection, където се предсказват bounding boxes, semantic segmentation връща плътна двумерна карта на класовете.

Тази задача е особено важна при автономно шофиране и анализ на градски сцени, защото системата трябва да разбира къде се намират пътят, тротоарът, автомобилите, пешеходците, знаците, светофарите и други обекти. В контекста на Cityscapes целта е да се класифицират пикселите към 19 тренировъчни класа.

3 Цел на проекта и изисквания

Основната цел е изграждане на end-to-end система за semantic segmentation на градски сцени. Системата трябва да позволява обучение, оценяване, сравнение и демонстрация

на различни модели.

Проектът покрива следните изисквания:

1. Проучване на научни статии и техники за semantic segmentation.
2. Сравнение на pixel-wise Cross Entropy loss, Dice loss и Focal loss.
3. Анализ на Cityscapes dataset: брой изображения, класове, class imbalance, аномалии и визуализации.
4. Експерименти с autoencoder-type архитектура, реализирана чрез Tiny U-Net.
5. Демонстрация на learning-rate scheduling чрез cosine annealing и step decay.
6. Експерименти с различни preprocessing и augmentation стратегии.
7. Streamlit web интерфейс за inference.
8. Unit tests за ключови компоненти.
9. Model report файл с baseline, експерименти, метрики, коментари и графики.

4 Dataset: Cityscapes

4.1 Използвани пакети

За проекта са нужни два основни Cityscapes пакета:

`leftImg8bit_trainvaltest.zip`

където са RGB изображенията за train, val и test

`gtFine_trainvaltest.zip`

където са фините pixel-level анотации за train и val

Очакваната структура на данните е:

```
data/raw/cityscapes/
+-- leftImg8bit/
|   +-- train/
|   +-- val/
|   +-- test/
+-- gtFine/
    +-- train/
    +-- val/
    +-- test/
```

4.2 Използване на split-овете

- **train** – използва се за обучение на моделите;
- **val** – използва се за сравнение на моделите и финална оценка;
- **test** – използва се само за inference/demo, защото публичният test split няма реални ground-truth masks.

4.3 Класове

В Cityscapes се използват 19 тренировъчни класа. Те са:

Пътна инфраструктура	Обекти / среда	Участници в движението
road	building	person
sidewalk	wall	rider
terrain	fence	car
	pole	truck
	traffic light	bus
	traffic sign	train
	vegetation	motorcycle
	sky	bicycle

Част от оригиналните Cityscapes label IDs се игнорират при обучение. За тази цел се използва `labelTrainIds` mapping и `ignore_index = 255` за невалидни или неизползвани пиксели.

5 Exploratory Data Analysis и подготовка на данните

EDA фазата има за цел да провери дали dataset-ът е консистентен и подходящ за обучение. Реализационният план изисква анализ на броя наблюдения, class distribution, missing image-mask pairs, invalid mask values и визуализации.

Основните проверки са:

- проверка на броя изображения и маски в **train** и **val**;
- проверка дали всяко изображение има съответна маска;
- проверка на размерите на image-mask двойките;

- проверка на валидните train IDs и 255 ignore пиксели;
- анализ на class imbalance;
- визуализация на image, ground truth mask и overlay.

5.1 Preprocessing

Използваната основна preprocessing конфигурация е:

- resize на изображението до 512x1024;
- random crop до 512x512;
- ImageNet normalization;
- синхронизирани геометрични трансформации върху image и mask;
- nearest-neighbor interpolation за маските.

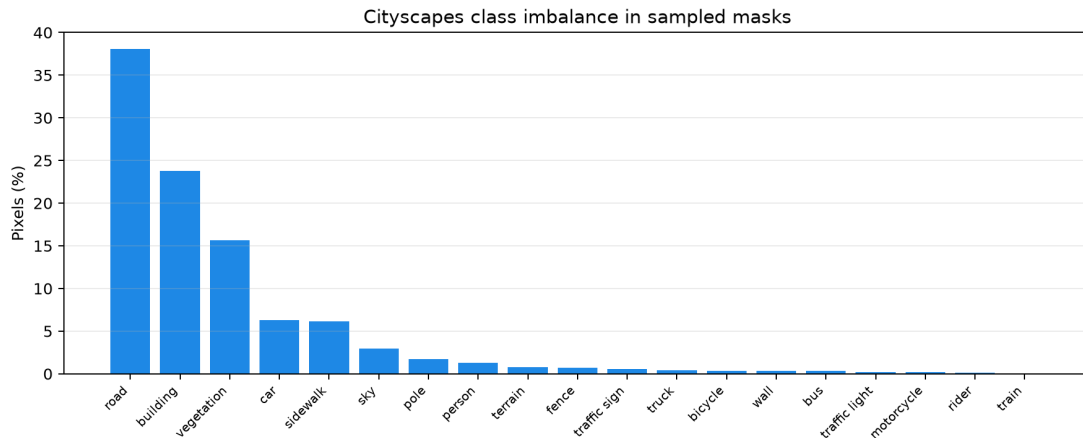
Важно е маските да не се нормализират като изображения. Те трябва да останат като цели числа, представящи класове.

5.2 Augmentation стратегии

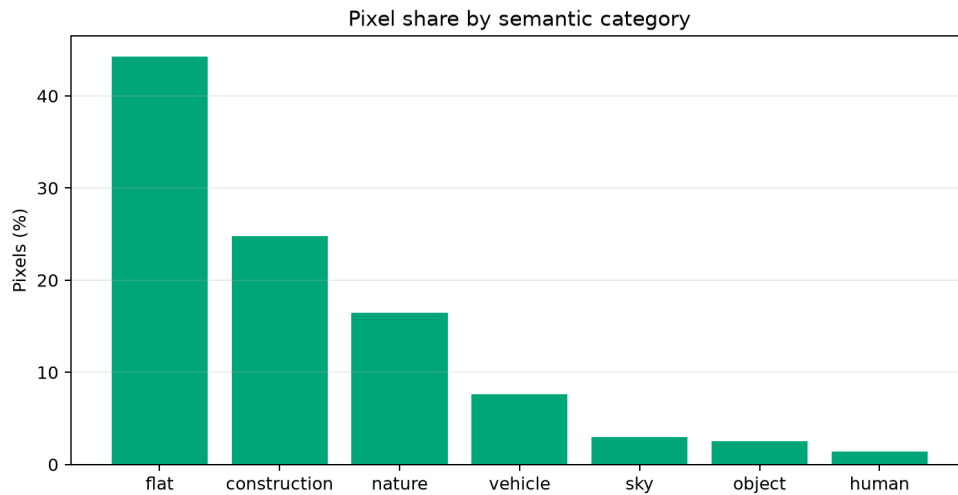
Тествани са няколко preprocessing/augmentation стратегии:

- `resize_only` – само преоразмеряване;
- `basic_aug` – resize/crop, horizontal flip, brightness/contrast;
- `strong_aug` – по-силни цветови и геометрични трансформации.

Изводът от експериментите е, че по-силните трансформации не винаги подобряват mean IoU. В някои случаи те влошават стабилността на validation резултатите.



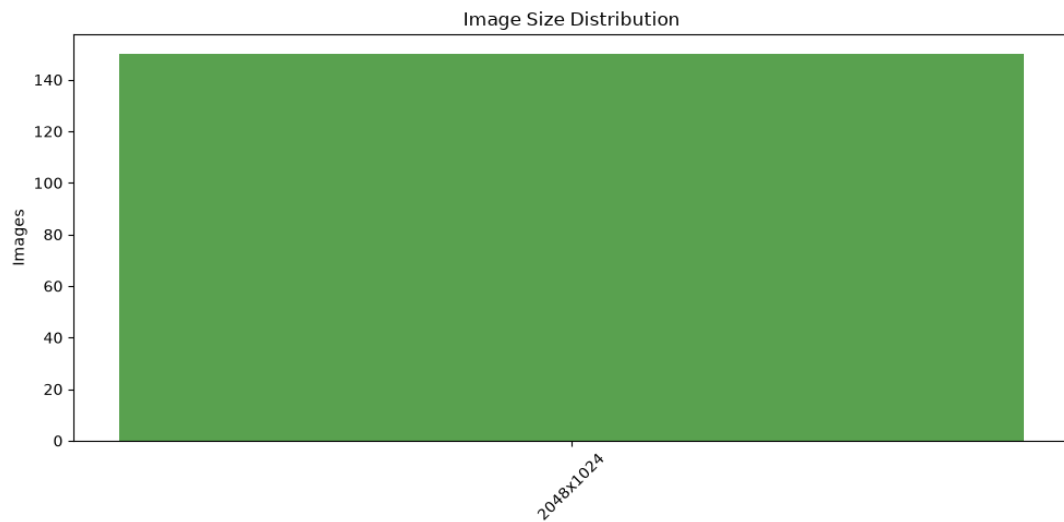
Фигура 1: Разпределение на пикселите по класове. Графиката показва силен class imbalance, доминиран от road, building и vegetation.



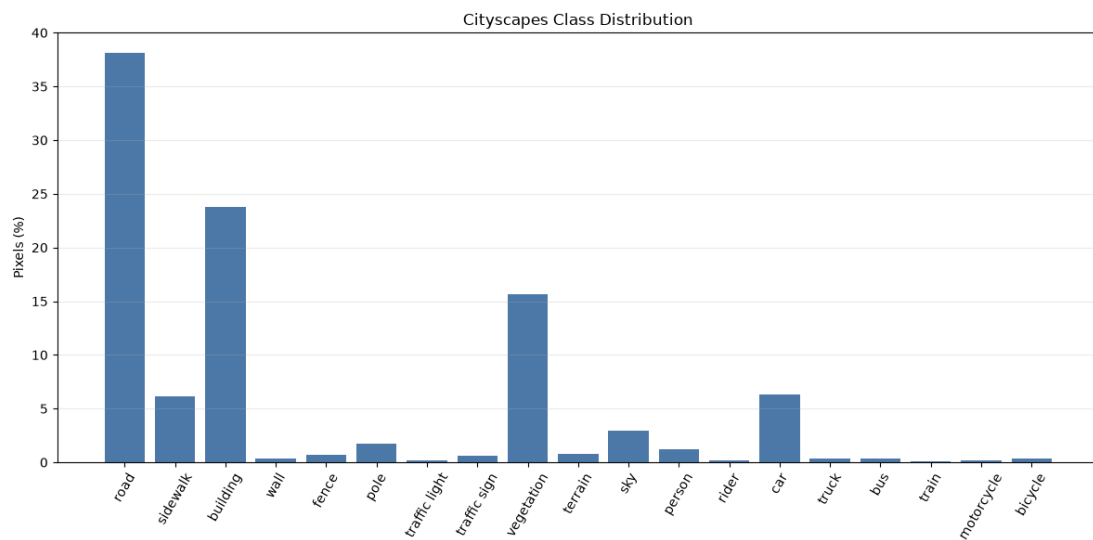
Фигура 2: Разпределение на пикселите по семантични категории. Flat, construction и nature категориите покриват най-голяма част от сцените.

5.3 EDA визуализации от reports/figures

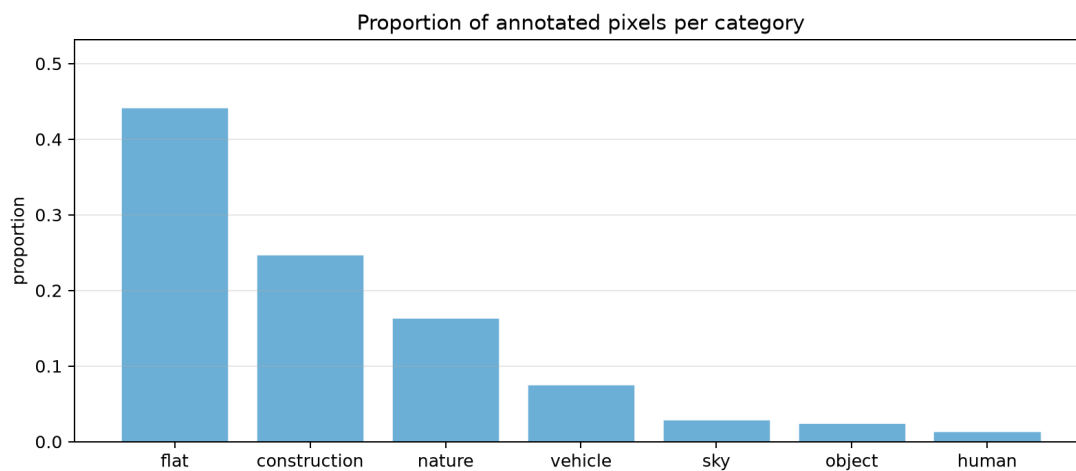
Следващите фигури са директно генерирани от `make eda` и са запазени в `reports/figures`. Те документират както количествените характеристики на dataset-a, така и качествени примери за image-mask съответствие.



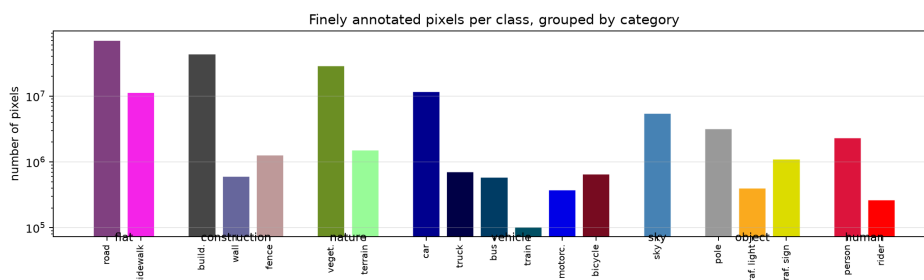
Фигура 3: Разпределение на размерите на изображенията в анализираната извадка. Cityscapes изображенията са консистентни като резолюция.



Фигура 4: Class distribution, генериран от EDA скрипта. Графиката потвърждава силния дисбаланс в полза на големите структурни класове.



Фигура 5: Дял на пикселите по семантични категории. Flat и construction категориите доминират в градските сцени.

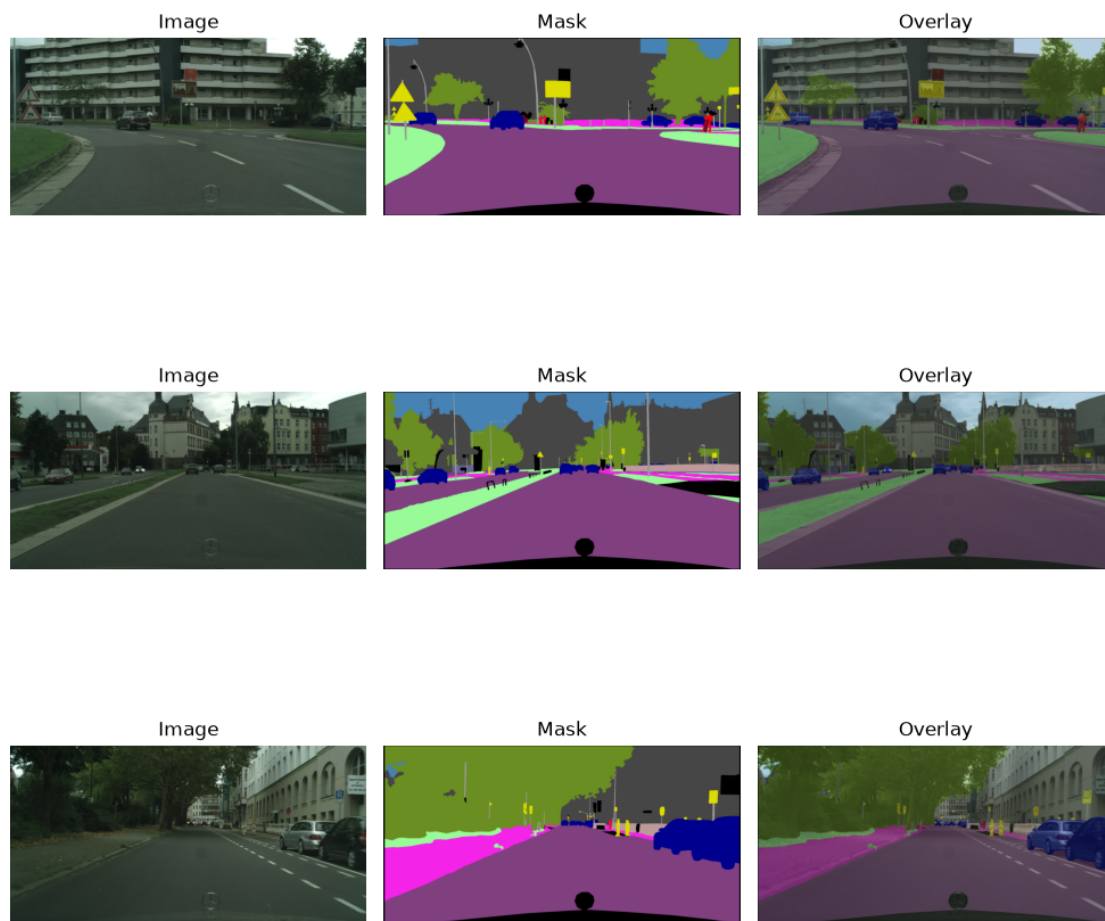


Фигура 6: Брой пиксели по класове, групирани по категории. Тази визуализация следва стила на dataset анализите в semantic segmentation статии.

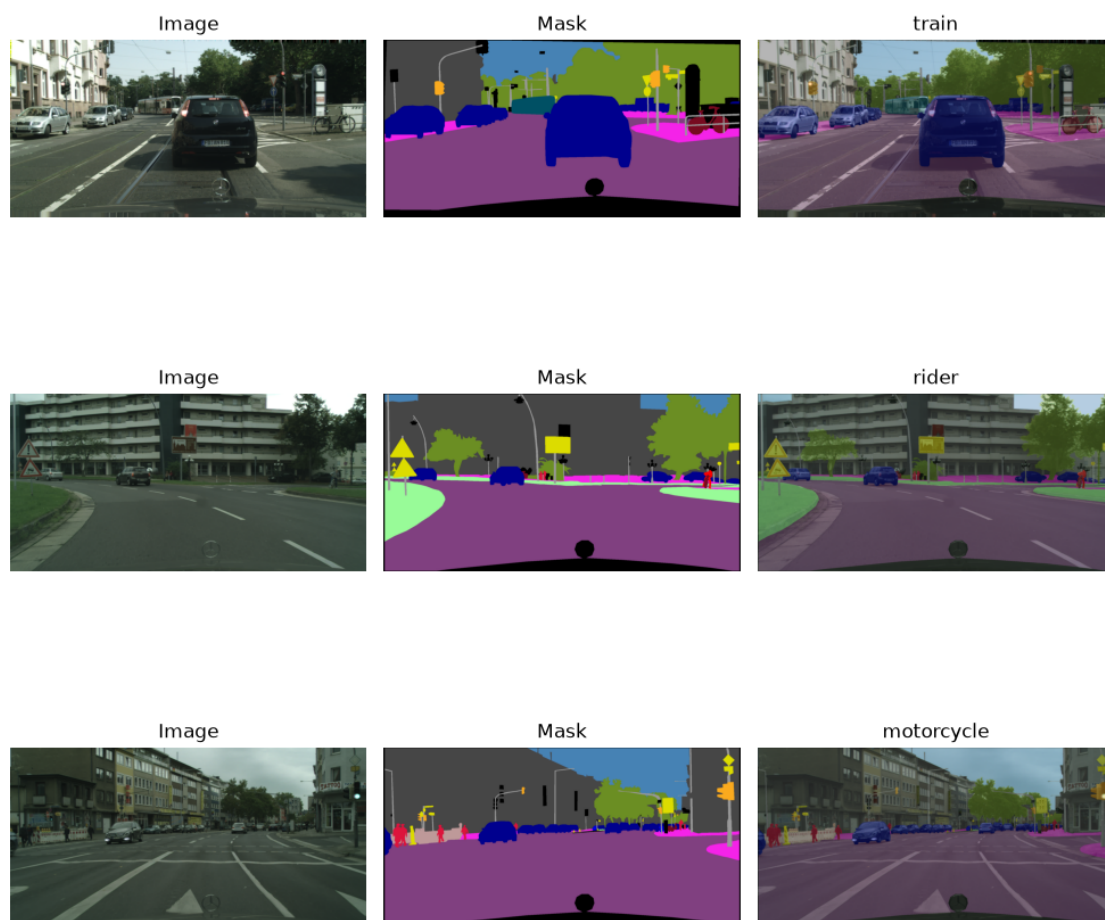
Dataset annotation summary

statistic	value
images	5,000
masks	5,000
sampled pixels	314,572,800
labeled trainId pixels	184,452,899
annotation density	58.64%
train classes	19

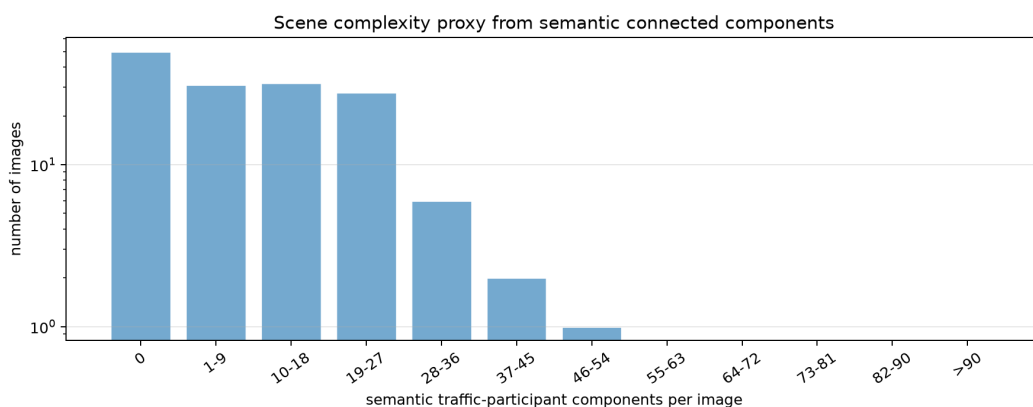
Фигура 7: Обобщение на аотациите и label density. Таблицата служи като compact EDA преглед на dataset-a.



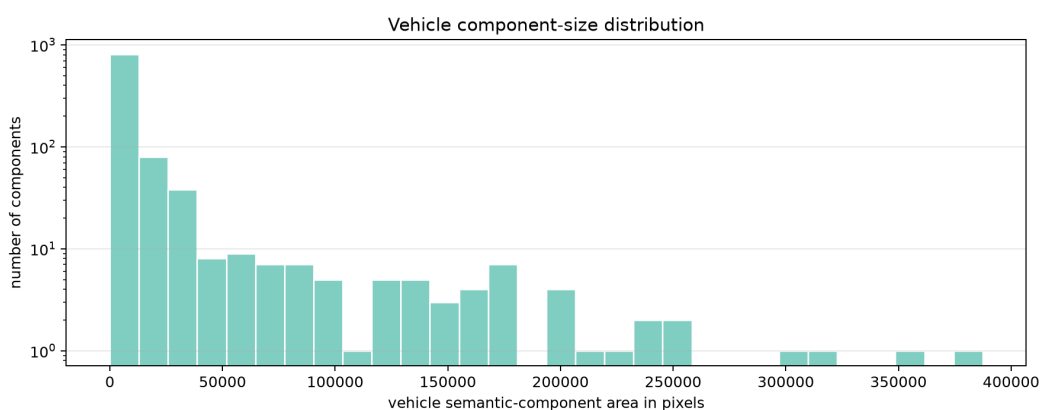
Фигура 8: Примерни image-mask overlays. Фигурата показва, че маските са правилно подравнени с RGB изображенията.



Фигура 9: Примери с редки класове. Тези изображения са важни за qualitative анализ, защото редките класове често са най-трудни за сегментиране.



Фигура 10: Оценка на scene complexity чрез traffic participant компоненти. По-сложните сцени съдържат повече малки и припокриващи се обекти.



Фигура 11: Хистограма на площите на vehicle компоненти. Малките компоненти обясняват защо car, truck, bus и motorcycle класовете изискват добра multi-scale обработка.

6 Training pipeline

Проектът използва конфигурируем training pipeline, управляван чрез YAML конфигурации. Това позволява повтораемост и ясно сравнение между експериментите.

Основните компоненти са:

1. Dataset loader за Cityscapes.
2. Preprocessing и augmentations.
3. Model factory за избор на архитектура.
4. Loss factory за избор на loss функция.
5. Optimizer и scheduler.

6. Training loop с mixed precision.
7. Evaluation loop с mean IoU, Dice и pixel accuracy.
8. Checkpoint saving.
9. Export на history, plots, confusion matrix и model report.

Примерна конфигурация:

```
training:
  epochs: 30
  batch_size: 4
  mixed_precision: true

model:
  architecture: deeplabv3plus
  encoder_name: resnet101
  encoder_weights: imagenet
  num_classes: 19

loss:
  name: cross_entropy_dice

scheduler:
  name: cosine_annealing
```

7 Използвани архитектури

7.1 Baseline majority-class model

Baseline моделът е greedy статистически модел, който предсказва най-често срещания клас за всеки валиден пиксел. В проекта този модел е `000_baseline_majority`.

Неговите резултати са:

- mean IoU = 0.0198;
- mean Dice = 0.0288;
- pixel accuracy = 0.3765.

Baseline е слаб, но е важен, защото всички невронни модели трябва да се сравняват спрямо него.

7.2 Tiny U-Net

Tiny U-Net е малка custom encoder-decoder архитектура, реализирана ръчно в PyTorch. Тя изпълнява изискването за autoencoder-type neural network. Архитектурата съдържа:

- encoder blocks с convolution, batch normalization и ReLU;
- max pooling за downsampling;
- bottleneck слой;
- decoder с upsampling или transposed convolution;
- skip connections чрез concatenation;
- final 1x1 convolution към 19 класа.

Tiny U-Net е полезен за валидиране на pipeline-а, но значително изостава спрямо pretrained encoder моделите.

7.3 U-Net

U-Net е класическа encoder-decoder архитектура за segmentation. Encoder-ът извлича все по-абстрактни признаци, а decoder-ът възстановява пространствената резолюция. Skip connections прехвърлят детайлна локална информация от encoder към decoder.

В проекта U-Net е използван с различни encoders:

- ResNet18;
- ResNet34;
- ResNet50;
- EfficientNet-B3.

Най-добрата U-Net конфигурация е 030_final_unet_resnet34_ce_dice_cosine, която постига mIoU = 0.6920.

7.4 U-Net++

U-Net++ разширява U-Net чрез по-гъсти skip connections между encoder и decoder блоковете. Целта е да се намали semantic gap между ниско-level feature maps и decoder feature maps.

В проекта `044_unetplusplus_resnet34_ce_dice` постига $mIoU = 0.6925$, което е малко над стандартния U-Net ResNet34 CE+Dice резултат.

7.5 FPN

Feature Pyramid Network (FPN) използва признаци от различни дълбочини на encoder-а и ги агрегира в multi-scale представяне. Това е особено полезно за Cityscapes, защото в една сцена има големи обекти като road/building и малки обекти като traffic light/sign.

Финалният най-добър модел е `046_fpn_efficientnet_b3_ce_dice` с $mIoU = 0.7283$.

7.6 PSPNet

PSPNet използва pyramid pooling за улавяне на глобален контекст. Идеята е моделът да разбира по-добре цялата сцена, а не само локалните детайли.

В проекта `024_final_pspnet_resnet34_ce_cosine` постига $mIoU = 0.5835$. Това е по-ниско от FPN, DeepLabV3+ и U-Net конфигурациите, но е полезен сравнителен експеримент.

7.7 DeepLabV3+

DeepLabV3+ комбинира atrous convolution, Atrous Spatial Pyramid Pooling (ASPP) и decoder модул. Atrous convolution позволява по-голям receptive field без директно намаляване на резолюцията.

Най-добрият DeepLabV3+ експеримент е `047_deeplabv3plus_efficientnet_b3_ce_dice` с $mIoU = 0.7256$. Той е много близък до най-добрия FPN модел.

7.8 SegFormer

SegFormer е transformer-based архитектура за semantic segmentation. Тя използва hierarchical transformer encoder и лек MLP decoder. В проекта `049_segformer_mit_b1_ce_dice_optional` е най-добрият transformer-based експеримент с $mIoU = 0.7087$.

8 Използвани encoders

8.1 ResNet

ResNet архитектурите използват residual connections, които улесняват обучението на по-дълбоки мрежи. В проекта са използвани ResNet18, ResNet34, ResNet50 и ResNet101.

Разликата между тях е основно в дълбочината и броя параметри. По-дълбоките модели могат да извличат по-богати признаци, но изискват повече изчислителни ресурси и не винаги дават пропорционално подобрене.

8.2 EfficientNet-B3

EfficientNet използва compound scaling, при който depth, width и resolution се увеличават балансирано. EfficientNet-B3 дава по-силни pretrained features от ResNet34 в най-добрите експерименти.

Най-добрите два резултата използват EfficientNet-B3 encoder:

- FPN + EfficientNet-B3: mIoU = 0.7283;
- DeepLabV3+ + EfficientNet-B3: mIoU = 0.7256.

8.3 MiT-B1

MiT-B1 е encoder от SegFormer архитектурата. Той използва transformer blocks и hierarchical feature representation. В проекта MiT-B1 дава добър резултат, но не надминава EfficientNet-B3 моделите.

9 Loss функции

9.1 Cross Entropy Loss

Pixel-wise Cross Entropy разглежда всеки пиксел като отделен classification пример. За multi-class segmentation моделът връща logits с форма $[B, C, H, W]$, а target маската е $[B, H, W]$.

$$L_{CE} = - \sum_{c=1}^C y_c \log(p_c)$$

Предимството на Cross Entropy е стабилното обучение. Недостатъкът е, че при силен class imbalance тя може да фаворизира доминиращите класове.

9.2 Dice Loss

Dice Loss оптимизира припокриването между predicted mask и ground truth mask.

$$Dice = \frac{2TP}{2TP + FP + FN}$$

Тя е полезна при segmentation, защото измерва overlap quality. Самостоятелно Dice Loss може да бъде по-нестабилна, затова в проекта най-добри резултати дава комбинацията Cross Entropy + Dice.

9.3 Focal Loss

Focal Loss намалява тежестта на лесните примери и се фокусира върху трудни пиксели и редки класове.

$$L_{focal} = -(1 - p_t)^\gamma \log(p_t)$$

В проекта Focal+Dice дава силни резултати, но не надминава CE+Dice.

9.4 Lovasz Loss

Lovasz е loss, ориентиран към IoU/Jaccard оптимизация. В проекта CE+Lovasz е тестван чрез 048_unet_resnet34_ce_lovasz, но резултатът не надминава CE+Dice.

10 Метрики за оценка

10.1 Pixel Accuracy

Pixel Accuracy измерва дела на правилно класифицираните пиксели:

$$PixelAccuracy = \frac{correct\ pixels}{all\ valid\ pixels}$$

Тази метрика е лесна за разбиране, но не е достатъчна сама по себе си, защото Cityscapes има доминиращи класове като road, building и sky. Модел може да има висока pixel accuracy, но слаб IoU върху редки класове.

10.2 Intersection over Union

IoU е основната метрика в проекта:

$$IoU = \frac{TP}{TP + FP + FN}$$

Mean IoU е средната стойност на IoU по класове. Тя е по-информативна от pixel accuracy при class imbalance.

10.3 Dice coefficient

Dice coefficient е overlap метрика, близка по идея до IoU:

$$Dice = \frac{2TP}{2TP + FP + FN}$$

В model report се използват максимум три основни метрики: mean IoU, mean Dice и pixel accuracy.

11 Експериментален план

Първоначалният implementation plan предвижда baseline, tiny U-Net, U-Net, DeepLabV3+, FPN, PSPNet, U-Net++ и опционален SegFormer. Всички експерименти се управляват чрез YAML configs, а резултатите се записват в model report.

В проекта са реализирани общо 28 evaluation експеримента плюс baseline. Това надвишава минималното изискване от 8 експеримента и позволява надеждно сравнение между архитектури, encoders, loss функции, scheduler-и и preprocessing варианти.

12 Model report и резултати

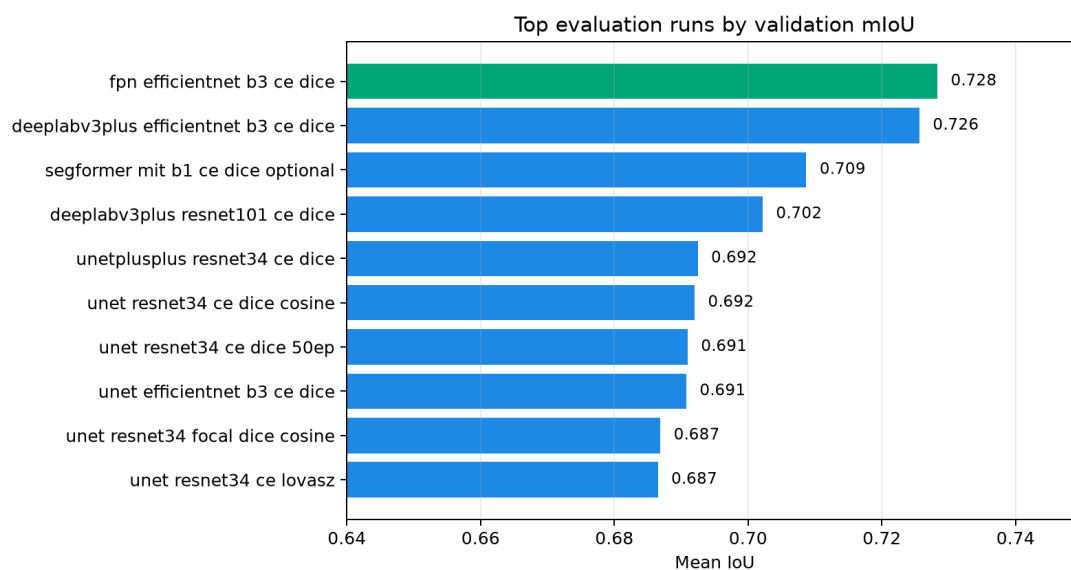
Model report файлът съдържа:

- baseline row като първи ред;

- всички evaluation експерименти;
- Top 10 highlight;
- процентна промяна спрямо baseline;
- коментари за всеки модел;
- loss curves за Top 5;
- artifacts/checkpoints sheet.

Таблица 2: Най-добри 10 evaluation експеримента, сортирани по mean IoU.

Rank	Experiment	Architecture	Encoder	Loss	mIoU	Dice	Acc.
1	046_fpn_efficientnet_b3_ce_dice	fpn	efficientnet-b3	cross_entropy_dice	0.7283	0.8335	0.9503
2	047_deeplabv3plus_efficientnet_b3_ce_dice	deeplabv3plus	efficientnet-b3	cross_entropy_dice	0.7256	0.8310	0.9482
3	049_segformer_mit_b1_ce_dice_optional	segformer	mit_b1	cross_entropy_dice	0.7087	0.8188	0.9488
4	043_deeplabv3plus_resnet101_ce_dice	deeplabv3plus	resnet101	cross_entropy_dice	0.7022	0.8130	0.9450
5	044_unetplusplus_resnet34_ce_dice	unetplusplus	resnet34	cross_entropy_dice	0.6925	0.8057	0.9495
6	030_final_unet_resnet34_ce_dice_cosine	unet	resnet34	cross_entropy_dice	0.6920	0.8066	0.9466
7	040_unet_resnet34_ce_dice_50ep	unet	resnet34	cross_entropy_dice	0.6910	0.8045	0.9478
8	045_unet_efficientnet_b3_ce_dice	unet	efficientnet-b3	cross_entropy_dice	0.6908	0.7946	0.9510
9	031_final_unet_resnet34_focal_dice_cosine	unet	resnet34	focal_dice	0.6869	0.8020	0.9456
10	048_unet_resnet34_ce_lovasz	unet	resnet34	cross_entropy_lovasz	0.6865	0.8006	0.9476



Фигура 12: Top evaluation runs, сортирани по validation mean IoU. Най-добрият модел е FPN + EfficientNet-B3 + CE+Dice.

13 Избор на най-добър модел

Най-добрият модел е 046_fpn_efficientnet_b3_ce_dice.

Компонент	Стойност
Architecture	FPN
Encoder	EfficientNet-B3
Encoder weights	ImageNet
Loss	Cross Entropy + Dice
Scheduler	Cosine Annealing
Epochs	30
Batch size	4
Image size / crop	512x1024 / 512x512
Augmentation	basic augmentation
Mean IoU	0.7283
Mean Dice	0.8335
Pixel Accuracy	0.9503

Този модел работи най-добре поради комбинацията от:

- multi-scale feature aggregation чрез FPN;
- силен pretrained EfficientNet-B3 encoder;

- CE+Dice loss, който балансира pixel-wise classification и overlap quality;
- Cosine Annealing scheduler, който стабилизира обучението.

14 Анализ на top моделите

14.1 FPN + EfficientNet-B3

FPN комбинира feature maps от различни нива на encoder-а. Това помага едновременно за големи класове като road и building и за по-малки обекти. EfficientNet-B3 предоставя по-силни представяния от ResNet34, което се вижда от най-високия mean IoU.

14.2 DeepLabV3+ + EfficientNet-B3

DeepLabV3+ с EfficientNet-B3 е вторият най-добър модел. Разликата спрямо FPN е малка, което показва, че atrous convolution и ASPP са много подходящи за Cityscapes.

14.3 SegFormer MiT-B1

SegFormer е най-добрият transformer-based run. Той се представя добре, но изостава от EfficientNet-B3 CNN моделите. Това може да се дължи на ограничен брой epochs, по-малък batch size или необходимост от по-фина настройка на learning rate.

14.4 U-Net и U-Net++

U-Net ResNet34 CE+Dice остава стабилен силен baseline. U-Net++ постига малко по-добър mIoU от стандартния U-Net, но не достига FPN и DeepLabV3+ с EfficientNet-B3.

15 Loss curves и generalization gap

За Top 5 моделите са анализирани train/validation loss curves. Основните наблюдения са:

- train loss намалява стабилно при всички Top 5 модела;
- validation loss също намалява, но остава по-висок от train loss;
- има умерен generalization gap, но не драматично overfitting поведение;

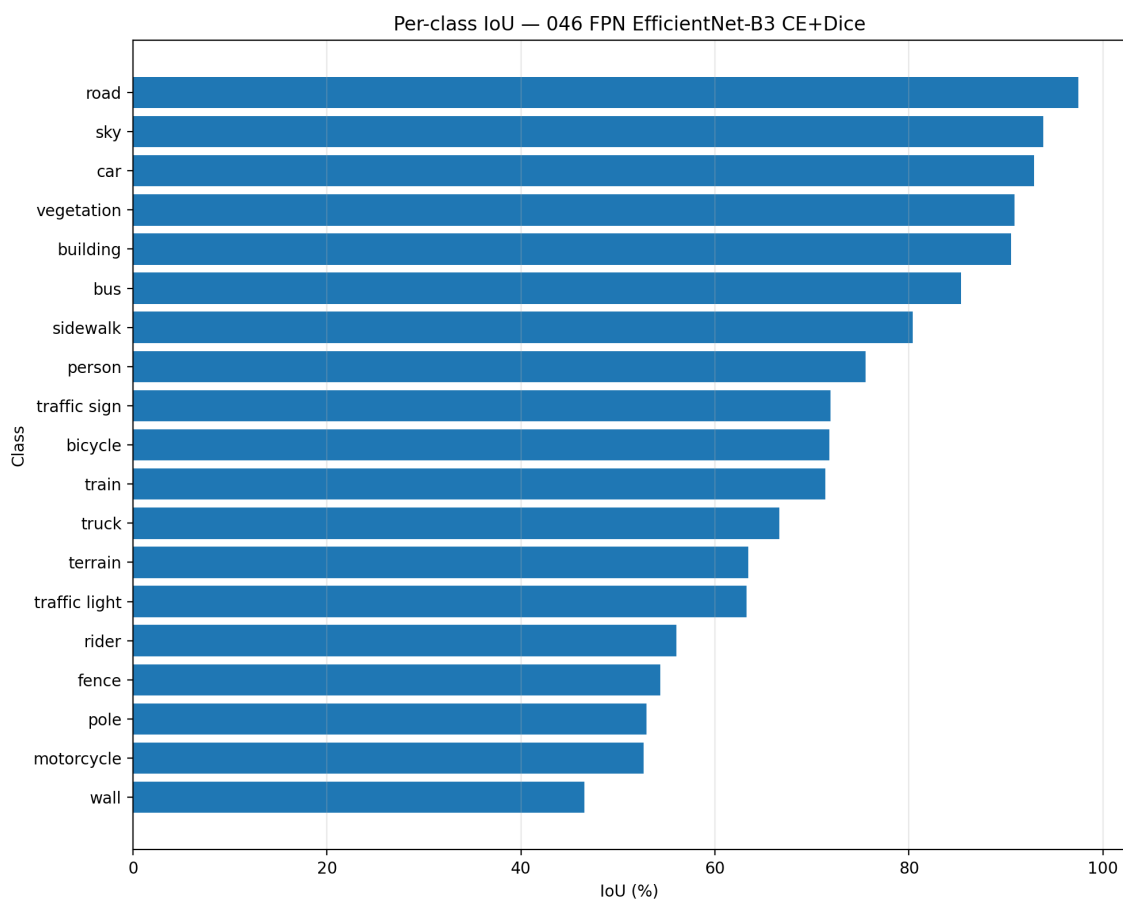
- при някои модели validation loss е по-шумен;
- Top 1 и Top 2 са много близки като качество.

16 Confusion matrix и error analysis

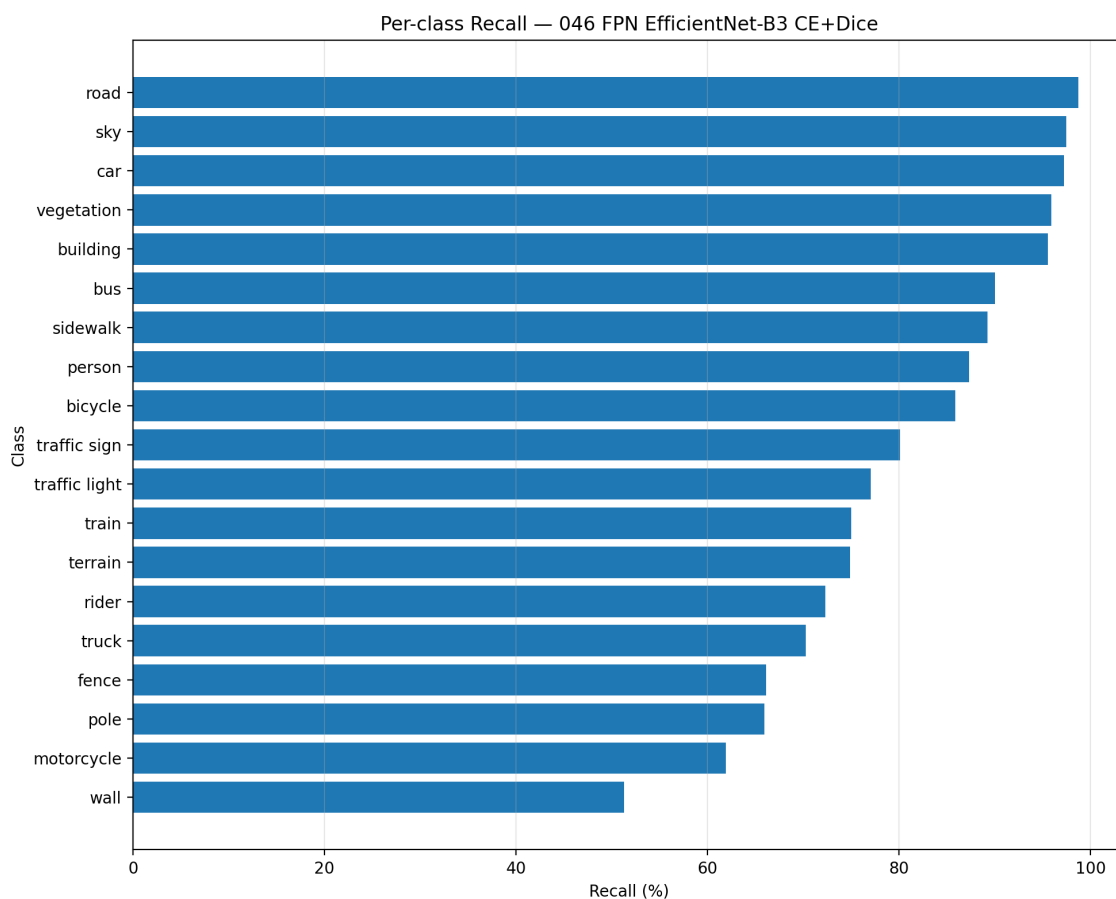
За най-добрия модел е изготвена confusion matrix визуализация. Общите изводи от анализа са:

- моделът е най-силен върху доминиращи и визуално стабилни класове като road, building, vegetation, car, sky и sidewalk;
- проблемните класове са малки, редки или boundary-sensitive;
- типични трудни класове са traffic light, traffic sign, rider, motorcycle и bicycle;
- чести грешки се появяват около граници между road, sidewalk и terrain;
- далечни и частично скрити обекти са по-трудни за сегментиране.

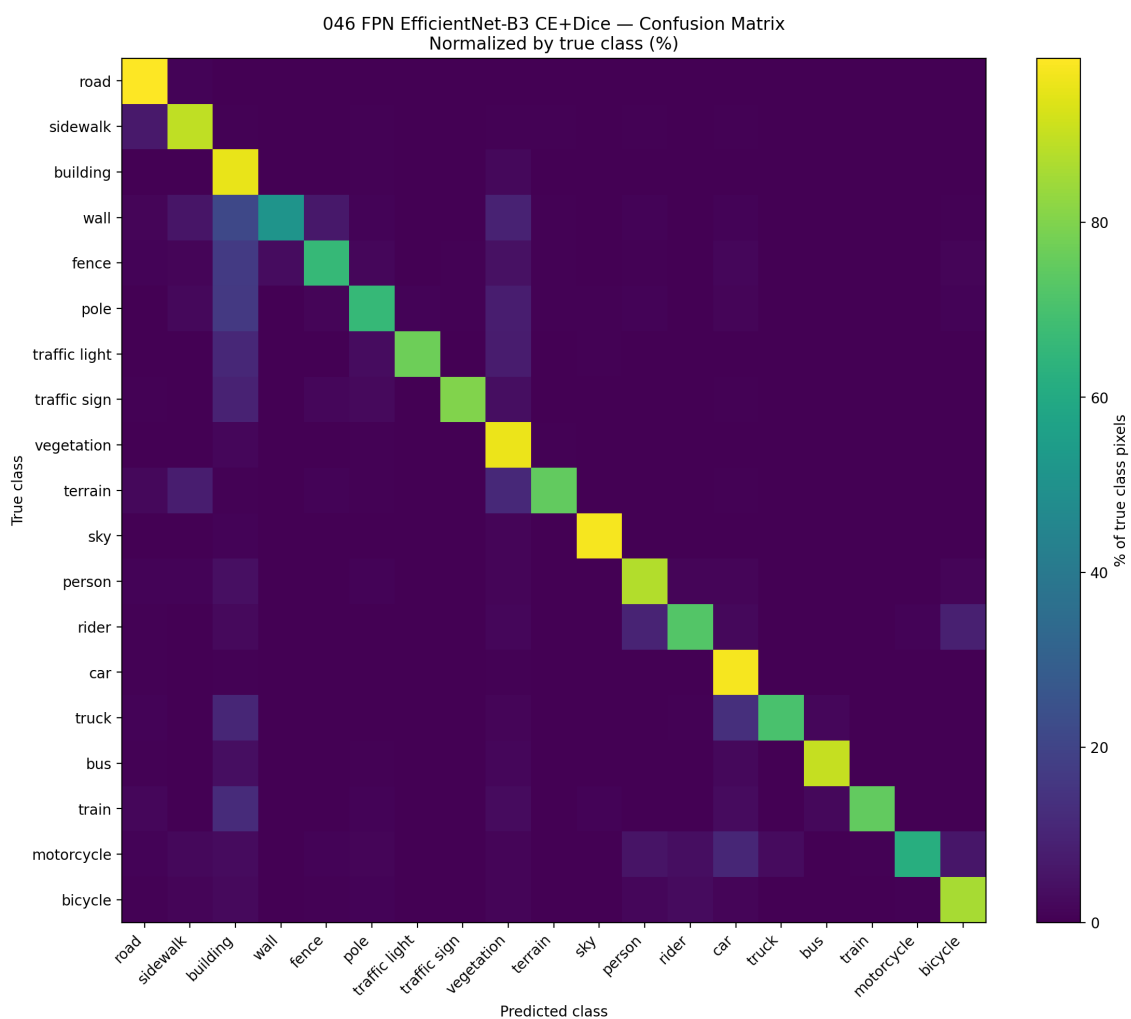
Това потвърждава защо mean IoU е по-подходяща основна метрика от pixel accuracy.



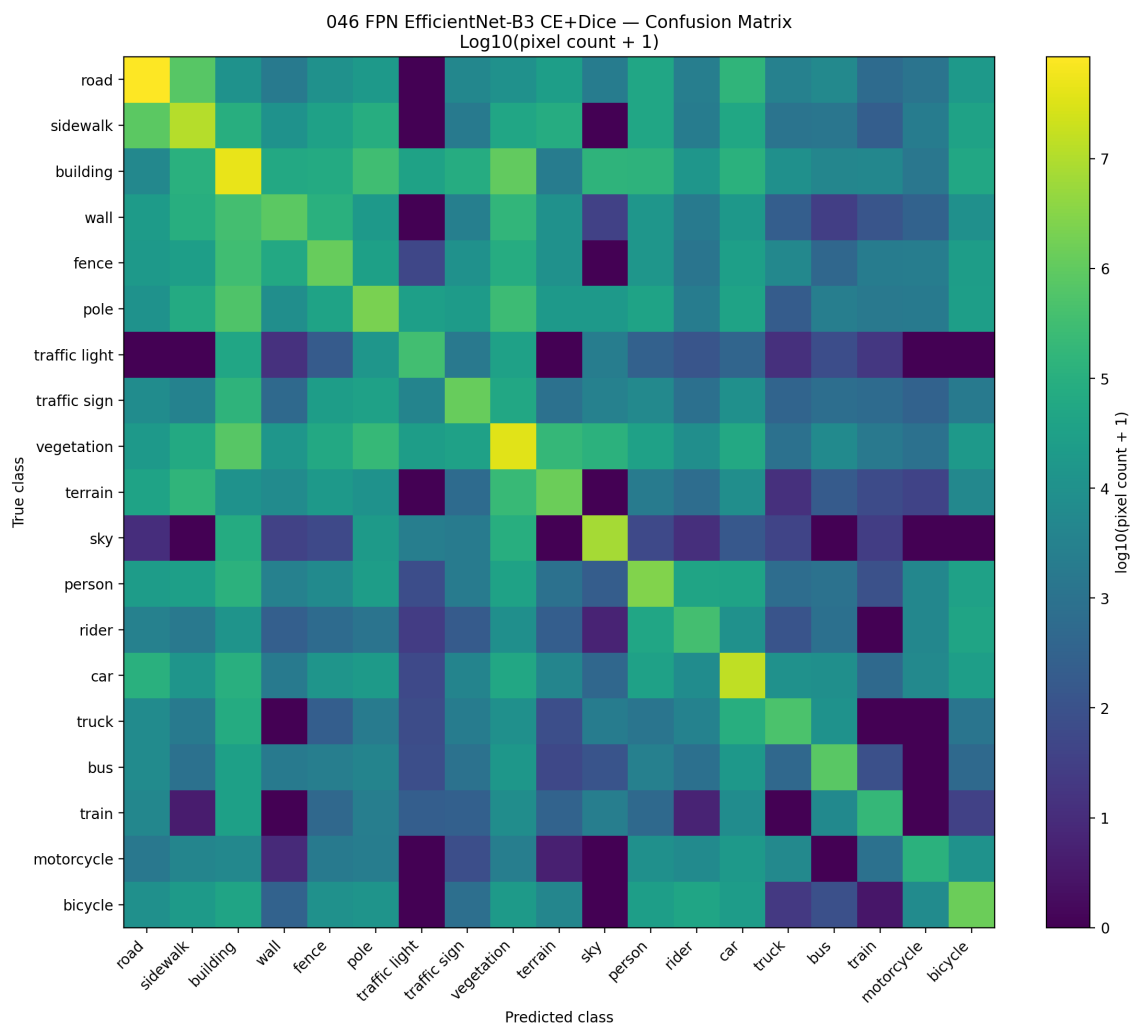
Фигура 13: Per-class IoU за най-добрия модел. Малките и редки класове имат по-нисък IoU от големите структурни класове.



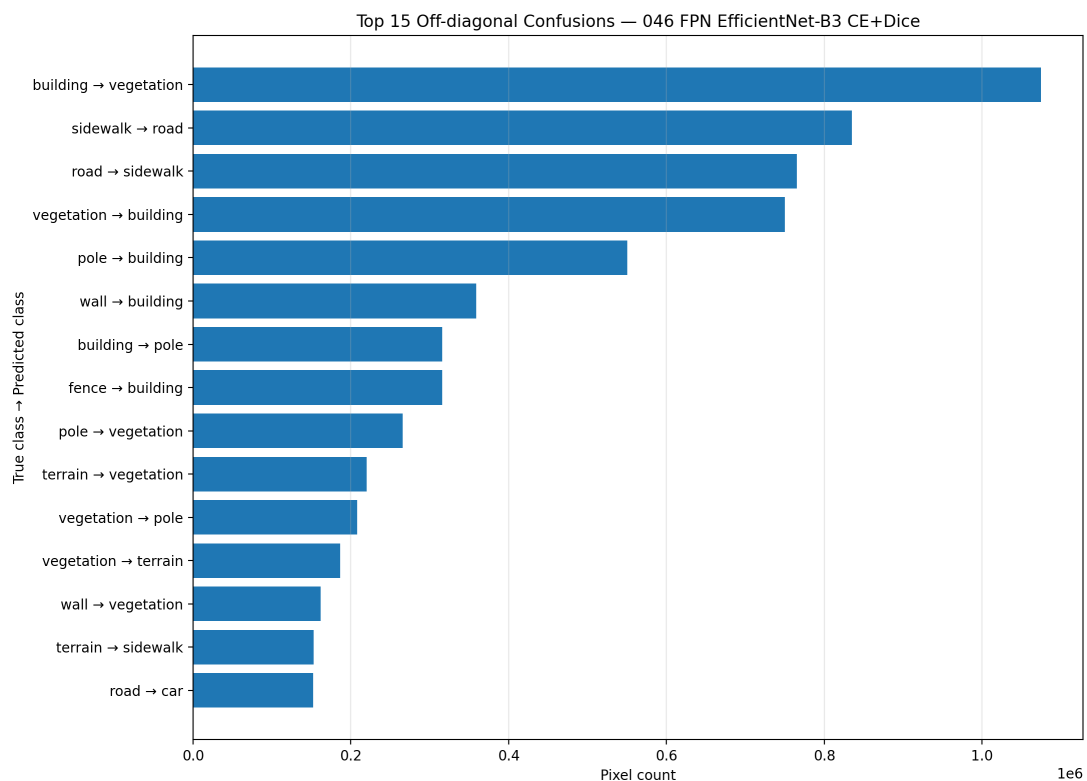
Фигура 14: Per-class recall за най-добрия модел. Recall анализът показва кои класове моделът пропуска най-често.



Фигура 15: Нормализирана confusion matrix по истински клас. Диагоналът показва правилните класификации, а off-diagonal стойностите показват най-честите обърквания.



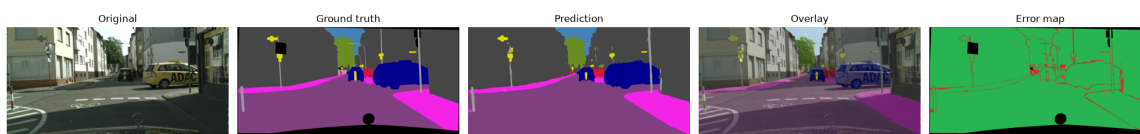
Фигура 16: Confusion matrix с логаритмична скала на броя пиксели. Логаритмичната визуализация прави видими и по-редките грешки.



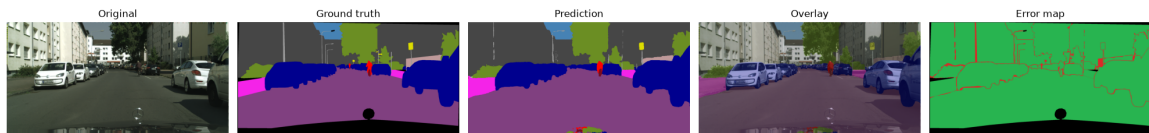
Фигура 17: Top 15 най-чести обърквания между класове. Тези грешки са основа за qualitative error analysis и бъдещи подобрения.

16.1 Qualitative validation examples

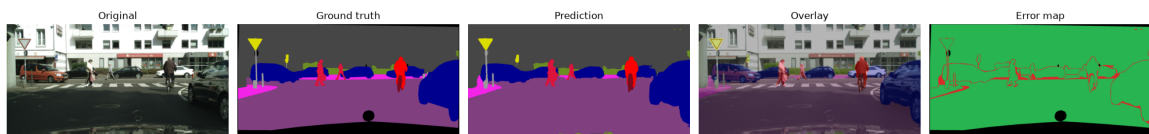
Следващите панели са генерирани от evaluation pipeline-a за най-добрия модел 046_fpn_efficientnet_b3_ce_dice. Всеки панел съдържа original image, ground truth, prediction, overlay и error map.



Фигура 18: Qualitative пример 1 за най-добрия модел. Панелът показва едновременно входното изображение, истинската маска, предсказанието, overlay и error map.



Фигура 19: Qualitative пример 2 за най-добрия модел. Error map частта позволява бърза визуална проверка на проблемните пиксели.



Фигура 20: Qualitative пример 3 за най-добрия модел. Такива примери допълват числените метрики с визуална интерпретация на грешките.

17 Streamlit demo приложение

Крайният проект включва Streamlit приложение за демонстрация на inference. Основните функционалности са:

- upload на изображение;
- избор на checkpoint;
- inference с избрания модел;
- показване на original image, predicted mask и overlay;
- легенда с класовете;
- опция за сравнение с ground truth, ако е наличен.

Streamlit приложението демонстрира практическата използваемост на проекта и затваря цикъла от обучение до визуална демонстрация.

18 Тестове и качество на кода

Implementation plan предвижда unit tests, които не изискват пълния Cityscapes dataset и могат да се изпълняват на CPU. Основните тестове са:

- label mapping тестове;
- dataset loading тестове със synthetic data;
- loss function тестове;
- metric тестове;
- model factory тестове;
- training step smoke test;
- inference и Streamlit import smoke tests.

Тези тестове гарантират, че pipeline-ът е възпроизводим и стабилен.

19 Ограничения

Проектът има няколко ограничения:

- тренировката е извършена върху resize/crop изображения, а не върху full-resolution Cityscapes изображения;
- някои архитектури са ограничени от GPU memory;
- публичният test split няма реални ground-truth masks;
- class imbalance остава предизвикателство;
- SegFormer експериментите са ограничени до MiT-B1;
- качественият анализ зависи от избрани validation примери.

20 Бъдеща работа

Възможни подобрения:

- class-weighted Cross Entropy;
- по-големи crops: 512x768 или 768x768;
- test-time augmentation;
- fine-tuning на най-добрия модел с по-нисък learning rate;
- SegFormer MiT-B2 или MiT-B3;
- DeepLabV3+ с други EfficientNet encoders;

- per-class confusion matrix и error analysis по класове;
- повече qualitative examples за добри и проблемни предсказания.

21 Заключение

Проектът демонстрира цялостен deep learning workflow за semantic segmentation: данни, preprocessing, модели, обучение, оценяване, model report и Streamlit demo.

Основният извод е, че **multi-scale feature aggregation**, комбиниран със **силен pretrained encoder** и **CE+Dice loss**, дава най-добро качество върху Cityscapes validation split. Най-добрият модел е FPN + EfficientNet-B3, който постига $\text{mean IoU} = 0.7283$.

Pixel Accuracy остава полезна допълнителна метрика, но не трябва да се използва самостоятелно, защото може да прикрие слабо представяне върху редки класове. Mean IoU е основната метрика за вземане на решение за най-добър модел.

А Пълен списък на експериментите

Таблица 4: Пълен списък на baseline и всички evaluation експерименти от model report файла.

Rank	Experiment ID	Model / Encoder	Loss	Scheduler	mIoU	Dice	Acc.
0	000_baseline_majority	majority_baseline / -	cross_entropy	cosine_annealing	0.0198	0.0288	0.3765
1	046_fpn_efficientnet_b3_ce_dice	fpn / efficientnet-b3	cross_entropy_dice	cosine_annealing	0.7283	0.8335	0.9503
2	047_deeplabv3plus_efficientnet_b3_ce_dice	deeplabv3plus / efficientnet-b3	cross_entropy_dice	cosine_annealing	0.7256	0.8310	0.9482
3	049_segformer_mit_b1_ce_dice_optional	segformer / mit_b1	cross_entropy_dice	cosine_annealing	0.7087	0.8188	0.9488
4	043_deeplabv3plus_resnet101_ce_dice	deeplabv3plus / resnet101	cross_entropy_dice	cosine_annealing	0.7022	0.8130	0.9450
5	044_unetplusplus_resnet34_ce_dice	unetplusplus / resnet34	cross_entropy_dice	cosine_annealing	0.6925	0.8057	0.9495
6	030_final_unet_resnet34_ce_dice_cosine	unet / resnet34	cross_entropy_dice	cosine_annealing	0.6920	0.8066	0.9466
7	040_unet_resnet34_ce_dice_50ep	unet / resnet34	cross_entropy_dice	cosine_annealing	0.6910	0.8045	0.9478
8	045_unet_efficientnet_b3_ce_dice	unet / efficientnet-b3	cross_entropy_dice	cosine_annealing	0.6908	0.7946	0.9510
9	031_final_unet_resnet34_focal_dice_cosine	unet / resnet34	focal_dice	cosine_annealing	0.6869	0.8020	0.9456
10	048_unet_resnet34_ce_lovasz	unet / resnet34	cross_entropy_lovasz	cosine_annealing	0.6865	0.8006	0.9476
11	023_final_fpn_resnet34_ce_cosine	fpn / resnet34	cross_entropy	cosine_annealing	0.6838	0.8000	0.9468
12	039_final_deeplabv3plus_resnet101_ce_cosine	deeplabv3plus / resnet101	cross_entropy	cosine_annealing	0.6793	0.7943	0.9480
13	025_final_deeplabv3plus_resnet50_ce_cosine	deeplabv3plus / resnet50	cross_entropy	cosine_annealing	0.6730	0.7887	0.9472
14	032_final_unet_resnet34_resize_only_ce_cosine	unet / resnet34	cross_entropy	cosine_annealing	0.6696	0.7871	0.9471
15	022_final_unetplusplus_resnet34_ce_cosine	unetplusplus / resnet34	cross_entropy	cosine_annealing	0.6686	0.7857	0.9486
16	033_final_unet_resnet34_strong_aug_ce_cosine	unet / resnet34	cross_entropy	cosine_annealing	0.6651	0.7835	0.9463
17	021_final_unet_resnet34_ce_cosine	unet / resnet34	cross_entropy	cosine_annealing	0.6648	0.7828	0.9469
18	038_final_unet_resnet50_ce_cosine	unet / resnet50	cross_entropy	cosine_annealing	0.6586	0.7763	0.9468
19	027_final_unet_resnet34_ce_plateau	unet / resnet34	cross_entropy	reduce_on_plateau	0.6558	0.7760	0.9446
20	029_final_unet_resnet34_focal_cosine	unet / resnet34	focal	cosine_annealing	0.6556	0.7752	0.9448
21	037_final_unet_resnet18_ce_cosine	unet / resnet18	cross_entropy	cosine_annealing	0.6398	0.7603	0.9436
22	026_final_unet_resnet34_ce_step	unet / resnet34	cross_entropy	step_decay	0.6037	0.7232	0.9419
23	024_final_pspnet_resnet34_ce_cosine	pspnet / resnet34	cross_entropy	cosine_annealing	0.5835	0.7101	0.9305
24	028_final_unet_resnet34_dice_cosine	unet / resnet34	dice	cosine_annealing	0.5488	0.6363	0.9308
25	036_final_tiny_unet_64_ce_cosine	tiny_unet / -	cross_entropy	cosine_annealing	0.3920	0.4841	0.8808
26	035_final_tiny_unet_32_ce_cosine	tiny_unet / -	cross_entropy	cosine_annealing	0.3437	0.4294	0.8657
27	020_final_tiny_unet_ce_cosine	tiny_unet / -	cross_entropy	cosine_annealing	0.3419	0.4265	0.8657
28	034_final_tiny_unet_16_ce_cosine	tiny_unet / -	cross_entropy	cosine_annealing	0.2610	0.3236	0.8323

Б Примерна структура на проекта

```
semantic-segmentation-cityscapes/  
+-- app/  
|   +-- streamlit_app.py  
+-- configs/  
|   +-- experiments/  
+-- data/  
|   +-- raw/cityscapes/  
+-- docs/  
+-- reports/  
+-- scripts/  
+-- src/cityseg/  
|   +-- data/  
|   +-- models/  
|   +-- training/  
|   +-- inference/  
|   +-- reporting/  
+-- tests/
```

В Използвана литература

1. Cordts et al., The Cityscapes Dataset for Semantic Urban Scene Understanding, CVPR 2016. URL: <https://arxiv.org/abs/1604.01685>
2. Long, Shelhamer, Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015. URL: <https://arxiv.org/abs/1411.4038>
3. Ronneberger, Fischer, Brox, U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015. URL: <https://arxiv.org/abs/1505.04597>
4. Noh, Hong, Han, Learning Deconvolution Network for Semantic Segmentation, ICCV 2015. URL: <https://arxiv.org/abs/1505.04366>
5. Chen et al., Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation, ECCV 2018. URL: <https://arxiv.org/abs/1802.02611>
6. Zhao et al., Pyramid Scene Parsing Network, CVPR 2017. URL: <https://arxiv.org/abs/1612.01105>
7. Xie et al., SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers, NeurIPS 2021. URL: <https://arxiv.org/abs/2105.15203>

8. Lin et al., Focal Loss for Dense Object Detection, ICCV 2017. URL: <https://arxiv.org/abs/1708.02002>
9. Milletari, Navab, Ahmadi, V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation, 2016. URL: <https://arxiv.org/abs/1606.04797>
10. Sudre et al., Generalised Dice Overlap as a Deep Learning Loss Function for Highly Unbalanced Segmentations, 2017. URL: <https://arxiv.org/abs/1707.03237>